Unidad 3 - Integración de contenido interactivo: jQuery

Curso 2026/2027

¿Qué es un Framework?

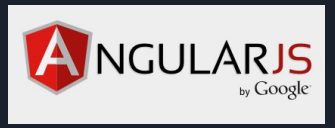
- **Framework**:
  - Literalmente es “**Marco de trabajo**”.
  - En desarrollo:
    - **Conjunto de librerías, metodologías y/o buenas prácticas** que tienen como objetivo facilitar el desarrollo de determinadas aplicaciones.
    - Los framework siempre están orientados a un propósito concreto:
      - Desarrollo de una parte del sistema:
        - Interfaz gráfico.
        - Persistencia.
        - ...
      - Desarrollo de sistemas concretos:
        - Sistemas de Información.
        - Juegos.
        - ...

Frameworks basados en JavaScript

- jQuery 

- React 
- Bootstrap 
- Backbone 

[Y muchos más]

- AngularJS 

- NodeJS 

- Google Web Toolkit 

- Dojo 

jQuery



- **Lema:**
  - **Write less, do more**.
- Creado en 2006.
- Amplia aceptación en la comunidad.
  - Muy bien documentado.
  - Desarrollo participativo de la comunidad.
  - Tiene gran cantidad de plug-ins para realizar tareas concretas.
    - Permite implementar test unitarios con QUnit.
- Licencia libre (MIT).
- Deriva el proyecto jQuery UI



¿Quién usa jQuery?



- El propio Google (A pesar de que tiene su librería).
- Microsoft.
- IBM
- Netflix
- DELL.
- mozilla.org
- Drupal.
- Wordpress.

















¡Muchos otros!

¿Se está quedando obsoleto?

- Mmmmm... Quizá para nuevos proyectos, aunque esto tiene matices y distintas opiniones.
- Actualmente hay otros frameworks que hacen lo mismo y de una manera más efectiva.
- Un claro ejemplo es que Bootstrap 5 (visto en el anterior tema) ha dejado fuera a jQuery, siendo capaz de realizar lo mismo y más que hacía en su versión anterior v4 (que sí incluía JQuery).
- Entonces, ¿por qué vamos a verlo y trabajarlo?



¿Se está quedando obsoleto?

- Aún hay **muchísimo** código
  que utiliza jQuery y que deberemos mantener.

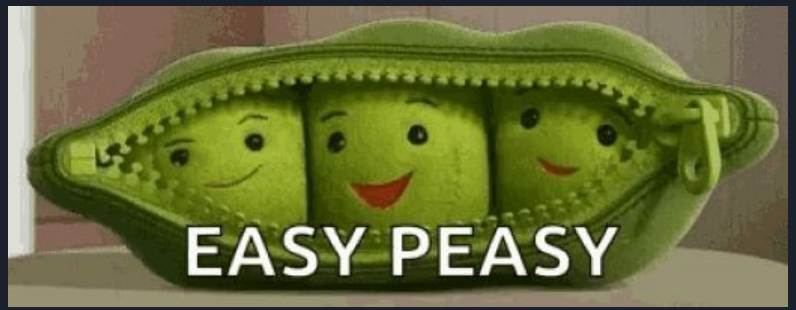
- Muchas empresas siguen utilizándolo.

  Ver estadística



¿Qué puedo hacer con jQuery?

- **Primordialmente**:
  - Acceder a los elementos de **DOM** de una manera más **sencilla** a la que proporciona JavaScript de manera estándar.
- Manejar de una forma más **sencilla eventos**.
- **Efectos especiales gráficos** de forma **sencilla**.
- **Validar formularios** de forma **sencilla**.
- Algunas utilidades básicas más.



Requisitos



- Para usar la librería es necesario:
  - Que el programador tenga conocimientos de:
    - HTML.
    - CSS → ¡Muy importante!
    - JavaScript.
  - Infraestructura técnica:
    - Cargar la librería en el navegador del cliente.
      - Debemos alojar la librería en nuestro servidor.
      - O usar una CDN (Content Distribution Network).

Versiones de la librería



- **Producción**:
  - Librería minimizada
    - Se compacta el código
    - Se denomina “minified” y su nombre lleva el sufijo “min”.
      - Ejemplo: ***jquery-3.7.1.min.js***

- Desarrollo:
  - Librería completa.
    - Viene bien para hacer depuración y ver qué está pasando

Inclusión de la librería jQuery



- La librería se ha de cargar en cada una de las páginas web de nuestro sitio:

```
<head>
    <script src="jquery-3.7.1.min.js"></script>
</head>
```

- O usando CDN:

```
<head>
    <script src="https://code.jquery.com/jquery-3.7.1.min.js"
        integrity="sha256-/JqT3SQfawRcv/BIHPThkBvs0OEvtFFmqPF/lYI/Cxo="
        crossorigin="anonymous"></script>
</head>
```

Hola mundo




```
<html>
<head>
    <title>jQuery Hello World</title>
    <script src="https://code.jquery.com/jquery-3.7.1.min.js"
        integrity="sha256-/JqT3SQfawRcv/BIHPThkBvs0OEvtFFm
    </script>
</head>
<body>
    <script>
        $(document).ready(function () {
            $("#msgid").html("This is Hello World by JQuery");
        });
    </script>
    <div id="msgid">
    </div>
</body>
</html>
```

Carga de librería

Espera que la página esté cargada

Localiza al elemento con id “msgid” y cambia su contenido interno

Llamadas a jQuery



- La librería jQuery se llama con el símbolo **dólar** y un par de paréntesis (es una función) → **$(...)**

```
$(document).ready(function(){
    $("#msgid").html("This is Hello World by JQuery");
});
```

  - Como argumento se pasa **el objeto** que queremos manipular **o un selector** para localizarlo.

- El equipo de jQuery también ha creado un método aún más corto para el evento de preparación del documento:

```
$(function(){
    $("#msgid").html("This is Hello World by JQuery");
});
```

¡Puede usarse cualquiera de las dos opciones!

Selectores



- Nos permite localizar elementos en la página:
  - Se usa una sintaxis similar a los selectores de HTML
    - Ejemplo:
      - “p”
      - “#msgid”.
      - “.clase”
    - Ejemplos más complejos:
      - “ol li”
      - “ol li.clase”

Selectores (Ejemplos)

Syntax	Description
$("*")	Selects all elements
$(this)	Selects the current HTML element
$("p.intro")	Selects all <p> elements with class="intro"
$("p:first")	Selects the first <p> element
$("ul li:first")	Selects the first element of the first
$("ul li:first-child")	Selects the first element of every
$("[href]")	Selects all elements with an href attribute
$("a[target='_blank']")	Selects all <a> elements with a target attribute value equal to "_blank"
$("a[target!='_blank']")	Selects all <a> elements with a target attribute value NOT equal to "_blank"
$(":button")	Selects all <button> elements and <input> elements of type="button"
$("tr:even")	Selects all even <tr> elements
$("tr:odd")	Selects all odd <tr> elements

Selectores (Referencia)



- **Oficial**:
  - http://api.jquery.com/category/selectors/

- No oficial:
  - Muy completa (Nuestros amigos de W3Schools).
    - http://www.w3schools.com/jquery/jquery_ref_selectors.asp



Eventos

- Un evento representa el momento preciso en que algo sucede.

  Ejemplos:

  - Al hacer clic o doble clic.
  - Al pasar el ratón por encima
  - ...

- Referencias:
  - http://api.jquery.com/category/Events/
  - http://www.w3schools.com/jquery/jquery_ref_events.asp

Efectos

- Facilitan realizar tareas gráficas:
  - Sencilla:
    - Desaparecer básico
  - Sencilla pero hay que pensar:
    - Alternar entre aparecer y desaparecer
  - No tan sencilla:
    - Desaparecer parametrizado (velocidad)



Efectos más comunes

Method	Description
animate()	Runs a custom animation on the selected elements
clearQueue()	Removes all remaining queued functions from the selected elements
delay()	Sets a delay for all queued functions on the selected elements
dequeue()	Removes the next function from the queue, and then executes the function
fadeIn()	Fades in the selected elements
fadeOut()	Fades out the selected elements
fadeTo()	Fades in/out the selected elements to a given opacity
fadeToggle()	Toggles between the fadeIn() and fadeOut() methods
finish()	Stops, removes and completes all queued animations for the selected elements
hide()	Hides the selected elements
queue()	Shows the queued functions on the selected elements
show()	Shows the selected elements
slideDown()	Slides-down (shows) the selected elements
slideToggle()	Toggles between the slideUp() and slideDown() methods
slideUp()	Slides-up (hides) the selected elements
stop()	Stops the currently running animation for the selected elements
toggle()	Toggles between the hide() and show() methods

Consulta y modificación de propiedades de elementos del DOM

- **Métodos**:
  - **text( )**: Devuelve el texto que hay dentro de un elemento (P. ej. el texto dentro de un párrafo).
  - **html( )**: Igual que el anterior pero devuelve también el HTML que hay dentro. Ej: **$("p").html("Hello world!");**
  - **val( )**: Permite manejar el valor de los campos de formularios.
  - **attr("atributo")**: Para obtener o modificar el valor de atributos.
- **Ejemplos:**
  - Ejemplo de consulta:
  - Ejemplo de modificación:
  - Ejemplos de atributos: Consulta y Modificación.

Adición de texto

- Métodos:
  - **Append**: Añade texto **al final** del elemento (dentro de él).
  - **Prepend**: Añade texto **al principio** del elemento (dentro de él).
  - **After y before**: Añade texto **antes** o **después** del elemento (fuera de él).

Eliminar contenido

- **Remove**:
  - Elimina **el elemento** y todo lo que contiene (su texto, hijos, etc.).
- **Empty**:
  - Elimina **el contenido** del elemento (sin eliminar el elemento en sí).

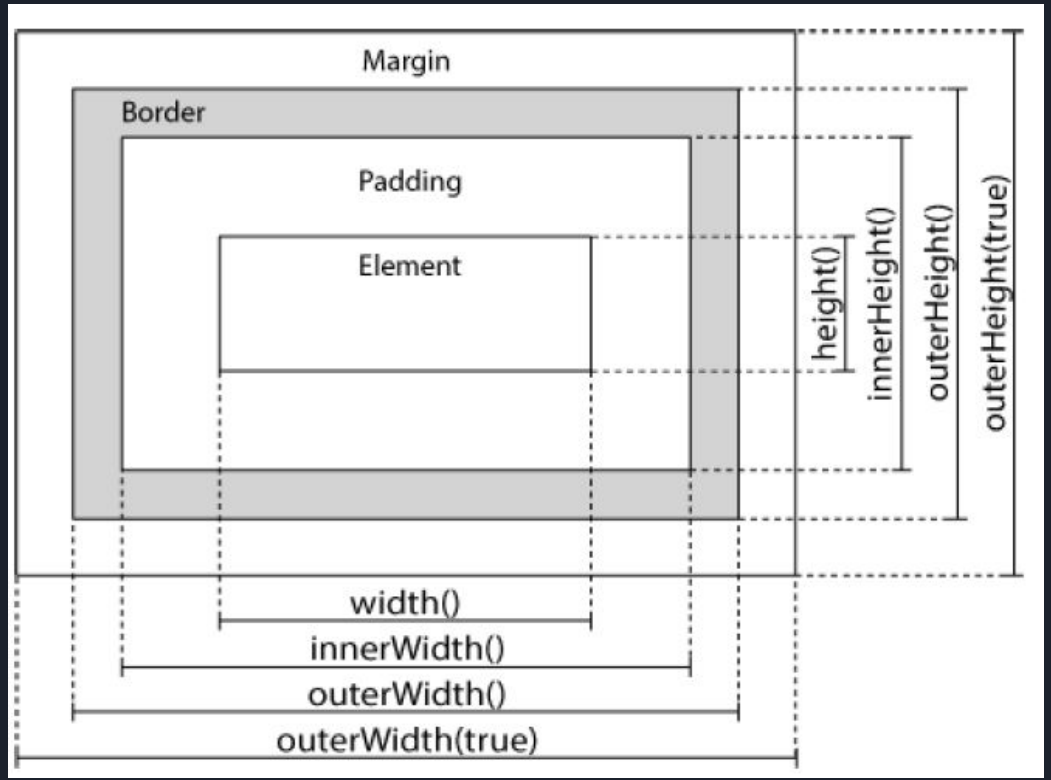
Modificaciones de CSS

- Añadir o eliminar una clase de un elemento.
- Variar el CSS de un elemento.



Tamaño de los elementos en jQuery

- width()
- height()
- innerWidth()
- innerHeight()
- outerWidth()
- outerHeight()
- Ejemplo



VALIDACIÓN SIMPLE DE FORMULARIOS CON JQUERY

- Vamos a programar un sencillo ejemplo de validación de campos utilizando jQuery.
- En ocasiones utilizamos formularios tan simples que no es preciso recurrir a otras librerías para su validación.
- Este es un ejemplo de ello, disponiendo además de un desarrollo reutilizable y adaptable a otros formularios.



EL FORMULARIO

Dispondremos de un formulario **muy simple** para realizar este ejemplo, con una estructura muy frecuente en cualquier página de contacto:

Un par de campos **<input>**, un **<select>**, un **checkbox**, una selección por **radiobutton,** un **<textarea>** para comentarios y un **botón de envío**.

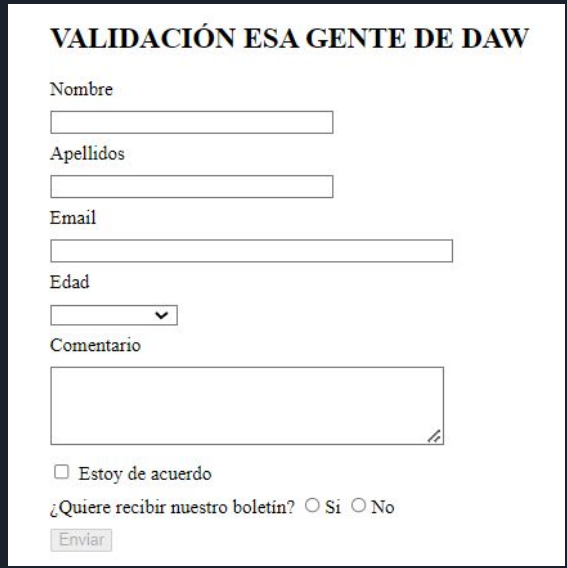


```html
<!DOCTYPE html>
<html>
<head>
    <title>VALIDACION ESA GENTE DE DAW </title>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" >
    <script src="https://code.jquery.com/jquery-3.7.1.min.js"
        integrity="sha256-/JqT3SQfawRcv/BIHPThkBvs0OEvtFFmqPF/lYI/Cxo="
        crossorigin="anonymous"></script>
    <link href="style.css" rel="stylesheet" type="text/css">
    <script type="text/javascript" src="validacion.js"></script>
</head>
<body>
    <section class="container">
        <h2>VALIDACIÓN ESA GENTE DE DAW </h2>
        <form name="contacto" id="contacto" method="post">
            <label for="nombre">Nombre</label>
            <input name="nombre" id="nombre" size="30" />
            <label for="apellidos">Apellidos</label>
            <input name="apellidos" id="apellidos" size="30" />
            <label for="email">Email</label>
            <input name="email" id="email" size="45" />
            <label for="edad">Edad</label>
            <select name="edad" id="edad">
                <option></option>
                <option value="18-25">18 a 25 años</option>
                <option value="26-45">26 a 45 años</option>
                <option value="46-65">46 a 55 años</option>
                <option value="66+">Más de 65</option>
            </select>
            <label for="comentario">Comentario</label>
            <textarea name="comentario" id="comentario" cols="40" rows="4"></textarea>
            <label for="legal">
                <input type="checkbox" name="legal" id="legal" /> Estoy de acuerdo
            </label>
            <label>¿Quiere recibir nuestro boletín?
                <input type="radio" name="boletin" value="si" />Si
                <input type="radio" name="boletin" value="no" />No
            </label>
            <button class="submit" disabled="true">Enviar</button>
        </form>
    </section>
</body>
</html>
```

EL FORMULARIO

style.css

```css
.container {
  margin: 0 auto;
  width: 90%;
  max-width: 1200px;
}
label {
  display: block;
  margin: .5em 0;
}
```



ESTRUCTURA DEL FORMULARIO

Como podemos ver, este formulario tiene una estructura muy simple, **asignando a cada elemento** del formulario **un *id*** con la misma cadena que el ***name***.

Una buena práctica: **asigna siempre que sea posible a cada elemento de un formulario un id para facilitar su localización por JS.**

*<form **name**="**contacto**" **id**="**contacto**" method="post">*

Otro de los aspectos a destacar es que el botón de **submit** se encuentra **desactivado** (disabled). El formulario no puede enviar la información hasta que nuestra validación compruebe que el formulario contiene los datos necesarios y mínimos para ser enviado.

<button class="submit" **disabled**>Enviar</button>

EL JAVASCRIPT

- Para facilitar la tarea de selección de los elementos HTML del formulario y la detección de eventos para comprobar el estado de cada uno de los campos utilizaremos como **base jQuery,** cargando desde un CDN la librería.
- A continuación incluiremos una serie de funciones que nos permitirán comprobar individualmente los datos que contienen cada uno de los campos del formulario, de modo que obtendremos como respuesta ***true*** cuando sea **válido** y ***false*** cuando **no cumpla** las condiciones necesarias.
- Para ello vamos a definir varias funciones distintas para los cuatro tipos básicos de elementos de formulario para introducir datos: **input, textarea, checkbox y radiobutton**. Estas funciones las añadiremos en el archivo **validacion.js**

VALIDACIÓN DE LOS INPUTS

- Para la validación de los inputs acudimos a las **expresiones regulares**.
- De este modo utilizaremos una **validación** para **Nombre y Apellidos** y otra para el input de **email**. Vamos a declarar varios patrones para cada tipo de campos. La función es más reutilizable, **pasándole** como parámetros el ***id*** del campo a comprobar **y la expresión regular** a utilizar para la validación.
- Para comprobar los campos input de Nombre y Apellidos nos valdremos del método ***.match()*** de **JavaScript**. En este ejemplo, para validar ambos campos, deberán de estar compuestos de una cadena de caracteres alfabéticos, con una longitud mínima de 4 caracteres y máxima de 30. Por supuesto admitiremos espacios. En caso de no cumplir esta condición devolverá un false.

VALIDACIÓN DE LOS INPUTS

En este ejemplo para validar ambos campos deberán de estar compuestos de una cadena de caracteres alfabéticos, con una longitud mínima de 4 caracteres y máxima de 30. Por supuesto admitiremos espacios. En caso de no cumplir esta condición devolverá un false.

```
var namePattern = "^[a-z A-Z]{4,30}$";
var emailPattern = "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+.[a-zA-Z]{2,4}$";

function checkInput(idInput, pattern) {
    return $(idInput).val().match(pattern) ? true : false;
}
```

Si la función ***match*** NO devuelve *null*, entonces devolveremos *true*, en cambio, si obtenemos *null*, devolveremos *false*.
Operador condicional.

En la validación del campo de email utilizamos un patrón mucho más complejo para asegurarnos que la cadena es una dirección de email.

COMPROBACIÓN DE TEXTAREA

En la función de comprobación de nuestro ***textarea*** somos menos rigurosos, aunque para validar este campo deberá tener una **extensión mínima de 13 caracteres**.

```
function checkTextarea(idText) {
return $(idText).val().length > 12 ? true : false;
}
```

Si la función ***length*** devuelve un valor mayor que 12, devolverá *true*, en caso contrario devolverá *false*.

COMPROBACIÓN DE RADIOBUTON Y CHECKBOXES

El método de comprobación de jQuery para **Radiobuton** y **Checkboxes** es el mismo, así que aprovecharemos dicho método para comprobar los dos tipos. Obviamente comprobamos como condición que el check relativo a la aceptación de nuestras normas es aceptado y que respondemos a la pregunta de si queremos suscribirnos al boletín o no.

```
function checkRadioBox(nameRadioBox) {
return $(nameRadioBox).is(":checked") ? true : false;
}
```

Por último comprobaremos que el campo **select** de Edad se encuentra cumplimentado también.

```
function checkSelect(idSelect) {
return $(idSelect).val() ? true : false;
}
```

ACTIVACIÓN Y DESACTIVACIÓN DEL SUBMIT

Por supuesto deberán de existir métodos o funciones que nos permitan **activar el botón de Enviar si** comprobamos que la **validación es correcta** y también una función que **desactive** el botón **si** ha cambiado algo en el formulario y ya **no cumple las condiciones** de envío.

```
function enableSubmit (idForm) {
$(idForm + " button.submit").removeAttr("disabled");
}

function disableSubmit (idForm) {
$(idForm + " button.submit").attr("disabled", "disabled");
}
```

VALIDACIÓN DEL FORMULARIO



- Por último nos encontramos con **el controlador de la validación**, que es quien comprueba el contenido del formulario ante los siguientes eventos: la modificación de un campo (**change**) o la pulsación de cualquier tecla (**keydown**).

- Si todas las condiciones que declaramos se cumplen, el botón Enviar se activará y podremos enviar el formulario. En caso contrario, es desactivado y no es posible enviarlo.

```
function checkForm (idForm) {
$(idForm + " *").on("change keydown", function() {
if (checkInput("#nombre", namePattern) &&
checkInput("#apellidos", namePattern) &&
checkInput("#email", emailPattern) &&
checkSelect("#edad") &&
checkTextarea("#comentario") &&
checkRadioBox("#legal") &&
checkRadioBox("[name=boletin]"))
{
enableSubmit(idForm);
} else {
disableSubmit(idForm);
}
});
 }
```

ACTIVACIÓN EN EL INICIO DEL FORMULARIO

Por supuesto, no podemos olvidarnos de **activar la validación del formulario** una vez cargado el documento, como cualquier plugin o desarrollo en jQuery. De este modo, permanecerá «**a la escucha**» ante cualquier cambio en el formulario para su validación.

```
$(function() {
checkForm("#contacto");
});
```

RESUMEN DEL CÓDIGO validacion.js

```
var namePattern = "^[a-z A-Z]{4,30}$";
var emailPattern = "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+.[a-zA-Z]{2,4}$";

function checkInput(idInput, pattern) {
	return $(idInput).val().match(pattern) ? true : false;
}
function checkTextarea(idText) {
	return $(idText).val().length > 12 ? true : false;
}
function checkRadioBox(nameRadioBox) {
  return $(nameRadioBox).is(":checked") ? true : false;
}
function checkSelect(idSelect) {
	return $(idSelect).val() ? true : false;
}
function enableSubmit (idForm) {
	$(idForm + " button.submit").removeAttr("disabled");
}
function disableSubmit (idForm) {
	$(idForm + " button.submit").attr("disabled", "disabled");
}
```

```
function checkForm (idForm) {
	$(idForm + " *").on("change keydown", function() {
		if (checkInput("#nombre", namePattern) &&
			checkInput("#apellidos", namePattern) &&
    checkInput("#email", emailPattern) &&
			checkSelect("#edad") &&
			checkTextarea("#comentario") &&
			checkRadioBox("#legal") &&
    checkRadioBox("[name=boletin]"))
		{
			enableSubmit(idForm);
		} else {
			disableSubmit(idForm);
		}
	});
}

$(function() {
	checkForm("#contacto");
});
```

CONCLUSIÓN

- Como hemos podido ver, es muy sencillo realizar cualquier validación con **jQuery,** dado que **el uso de selectores simplifica mucho la tarea**, así como **determinar los eventos** que lanzarán la validación.
- Lo más importante es determinar la validación de modo individual para cada tipo de formulario o cambio y poder tener un desarrollo lo más modular posible.
- En este caso creo que es un desarrollo bastante "**reaprovechable**" y espero os sirva para practicar y poder elaborar vuestros propios validadores.